

FIȘĂ DE LUCRU - Subprograme în Python

Informatică • Clasa a IX-a • Python

Fișe de lucru - Informatică, clasa a IX-a | Python

Date generale

Disciplina: Informatică | **Clasa:** a IX-a | **Durată:** 50 minute | **Tip:** fișă de lucru – teorie, exemplu, problemă

Structura unui subprogram în Python

def	cuvânt cheie folosit pentru definirea unei funcții
nume(...)	identificatorul funcției și lista de parametri
parametri	datele primite de funcție pentru prelucrare
return	instrucțiune care trimite rezultatul înapoi către program
apel	executarea funcției folosind argumente: <code>rezultat = nume(argumente)</code>

Tipul 1. Definirea și apelul unei funcții cu parametri

Partea teoretică. Un subprogram, numit și funcție, grupează instrucțiuni care rezolvă o sarcină clară. În Python, o funcție se definește cu `def`, primește date prin **parametri** și poate trimite un rezultat prin `return`.

⚠ Atenție!

- Parametrii apar la definirea funcției.
- Argumentele sunt valorile transmise la apel.
- În această fișă, accentul cade pe funcții care primesc date prin parametri.

✓ Exemplu rezolvat – Calculați consumul mediu al unui autoturism.

Funcția primește distanța parcursă și cantitatea de combustibil consumată, apoi returnează consumul la 100 km.

```
def consum_mediu(distanța, combustibil):  
    # calculăm consumul pentru 100 km  
    consum = combustibil / distanța * 100  
    return round(consum, 2)  
  
rezultat = consum_mediu(350, 21)  
  
print("Consum mediu:", rezultat, "l/100 km")
```

➡ Problemă propusă – Estimați durata unei călătorii.

- scrieți funcția `durata_calatorie(distanța, viteza)`;
- funcția returnează durata exprimată în ore;
- apelați funcția pentru două trasee diferite și afișați rezultatele.

Indicație: durata se calculează cu formula `distanța / viteza`.

Tipul 2. Parametri și instrucțiunea return

Partea teoretică. Parametrii permit transmiterea valorilor către funcție la momentul apelului. Instrucțiunea `return` trimite rezultatul calculat înapoi în locul din care funcția a fost apelată, astfel încât acesta să poată fi salvat, afișat sau folosit în alte calcule.

⚠ Atenție!

- `print()` doar afișează un mesaj pe ecran.
- `return` trimite rezultatul către program.
- O funcție cu `return` poate fi folosită în expresii și calcule.

✓ Exemplu rezolvat – Calculați nota finală a unui elev din trei componente.

Funcția primește nota de teorie, nota de practică și nota de proiect și returnează media ponderată: 40%, 40%, 20%.

```
def nota_finala(teorie, practica, proiect):
    # calculăm media ponderată a celor trei componente
    medie = teoria * 0.4 + practica * 0.4 + proiect * 0.2
    return round(medie, 2)

# citim datele de la tastatură
t = float(input("Nota teorie: "))
p = float(input("Nota practică: "))
pr = float(input("Nota proiect: "))

final = nota_finala(t, p, pr)

print("Nota finală:", final)

if final >= 5:
    print("Promovat!")
else:
    print("Nepromovat.")
```

➡ Problemă propusă – Calculați costul total al unui coș de cumpărături.

Scrieți funcția `cost_total(pret, cantitate, reducere)` care returnează prețul final după aplicarea reducerii procentuale. Testați funcția pentru cel puțin două produse.

Indicație: `cost = pret * cantitate * (1 - reducere / 100)`.

Tipul 3. Funcții cu mai mulți parametri

Partea teoretică. O funcție poate primi mai mulți parametri atunci când are nevoie de mai multe informații pentru a rezolva problema. Ordinea argumentelor de la apel trebuie să corespundă ordinii parametrilor din definiție.

⚠ Atenție!

- Numărul argumentelor trebuie să fie același cu numărul parametrilor.
- Valorile sunt preluate în ordinea în care sunt transmise la apel.

✓ Exemplu rezolvat – Determinați valoarea finală a unei facturi.

Funcția primește prețul unui produs, cantitatea cumpărată și TVA-ul, apoi returnează valoarea finală.

```
def factura(pret, cantitate, tva):  
    # calculăm valoarea fără TVA  
    subtotal = pret * cantitate  
  
    # adăugăm TVA-ul procentual  
    total = subtotal * (1 + tva / 100)  
  
    return round(total, 2)  
  
valoare = factura(35, 4, 19)  
  
print("Total factură:", valoare, "lei")
```

➔ Problemă propusă – Calculați tariful pentru o comandă livrată.

Scrieți funcția `pret_livrare(pret_produș, cantitate, taxa_livrare)` care returnează valoarea totală a unei comenzi, incluzând taxa de livrare.

- dacă valoarea produselor este cel puțin 300 lei, taxa de livrare devine 0;
- altfel, se adaugă taxa de livrare primită ca parametru.

Indicație: folosiți o instrucțiune `if` în interiorul funcției.

| Tipul 4. Aplicație modulară cu mai multe funcții

Partea teoretică. Proiectarea modulară înseamnă împărțirea unei probleme în subprobleme mai mici, fiecare rezolvată printr-o funcție cu rol clar. Astfel, programul devine mai ușor de citit, testat și modificat.

⚠ Atenție!

- Fiecare funcție ar trebui să aibă un rol bine definit.
- Funcțiile de prelucrare primesc date prin parametri și returnează rezultate.
- Funcția de citire poate avea rolul separat de a prelua datele introduse de utilizator.

✓ Exemplu rezolvat – Aplicație modulară pentru analiza notelor unei clase.

Separăm citirea datelor, calculul statisticilor și afișarea rezultatelor în funcții distincte.

```
def citeste_note():
    # citim notele pe aceeași linie, separate prin spațiu
    return [int(x) for x in input("Note: ").split()]

def calculeaza_statistici(note):
    # determinăm minimul, maximul și media listei de note
    minim = min(note)
    maxim = max(note)
    medie = round(sum(note) / len(note), 2)
    return minim, maxim, medie

def note_sub_medie(note, medie):
    # construim lista notelor mai mici decât media clasei
    return [n for n in note if n < medie]

def afiseaza_raport(minim, maxim, medie, sub_medie):
    # afișăm rezultatele calculate de celelalte funcții
    print("--- Raport ---")
    print("Minim:", minim, "| Maxim:", maxim, "| Medie:", medie)
    print("Note sub medie:", sub_medie)

# programul principal coordonează funcțiile
note = citeste_note()
mi, ma, med = calculeaza_statistici(note)
sub = note_sub_medie(note, med)
afiseaza_raport(mi, ma, med, sub)
```

➡ Problemă propusă – Aplicație modulară pentru gestionarea cheltuielilor lunare.

Scrieți o aplicație cu funcții separate pentru:

- citirea cheltuielilor;
- calculul totalului și al mediei;
- identificarea cheltuielilor peste medie;
- afișarea unui raport final.

Indicație: urmați structura din exemplu. Funcția de statistici poate returna totalul și media.

Autoevaluare

- ☐ Pot defini o funcție cu parametri.
- ☐ Pot apela o funcție folosind argumente.
- ☐ Înțeleg diferența dintre `print()` și `return`.
- ☐ Pot crea funcții cu mai mulți parametri.
- ☐ Pot organiza o aplicație în mai multe funcții cu roluri clare.

Temă opțională

Alegeți una dintre problemele propuse și scrieți programul complet în Python. Testați programul pentru:

- un caz obișnuit;
- un caz cu valori extreme sau repetate.